MQTT — Message Queuing Telemetry Transport

Reading an IIoT telemetry conversation, frame by frame

- **Protocol:** MQTT
- **Transport:** TCP/1883 (plaintext) · TCP/8883 (MQTT over TLS)
- **Reference:** OASIS MQTT Version 3.1.1
- **Capture file:** `mqtt_iot_telemetry.pcap`
- **Level:** Protocol analysis & defensive recognition — not an offensive tradecraft course. Intermediate (TCP/IP + basic Wireshark; new to IoT/OT)

Every frame number, field value, and CRC below was produced and verified with Wireshark/tshark and CISA's ICSNPP / Zeek. The capture is a curated teaching file — synthetic but protocol-valid.

1. What this module is

MQTT is the lightweight publish/subscribe protocol behind an enormous amount of the Internet of Things and Industrial IoT (IIoT). Instead of clients talking directly to each other, every device connects to a central **broker**. Publishers send messages to named **topics**; subscribers ask the broker for topics they care about; the broker fans each message out to whoever subscribed. A field sensor never needs to know who — if anyone — is listening.

That decoupling is what makes MQTT scale to millions of devices over flaky, low-bandwidth links. It also concentrates risk at the broker: whoever can reach it, and whatever the broker is (mis)configured to allow, defines the security of the whole system. MQTT 3.1.1 itself provides only optional username/password authentication — sent in cleartext unless you add TLS — and **no authorization model at all**. Topic access control is left entirely to the broker.

You will analyze `mqtt_iot_telemetry.pcap`, a curated 61-frame capture of a small plant telemetry deployment: an HMI dashboard subscribes, a field sensor publishes tank readings, and the broker fans them out. Then a rogue host connects anonymously, subscribes to everything, and injects a command. Every frame number and field in this module was produced and verified with Wireshark/tshark and Zeek's MQTT analyzer.

2. Learning objectives

1. Explain the MQTT publish/subscribe model and the broker's central role, and predict which clients receive a given published message from the subscription table.
2. From a capture, identify the core control packets (CONNECT, CONNACK, SUBSCRIBE, SUBACK, PUBLISH, PUBACK, PINGREQ/PINGRESP, DISCONNECT) and extract connection details from a CONNECT (client ID, clean session, keep-alive, username/password, Last Will), reading cleartext credentials directly off the wire.

3. Trace a single reading from publisher through the broker to every subscriber, and explain how QoS and the RETAIN flag change what a newly-connecting client receives, distinguishing a live PUBLISH from a delivered retained message by the RETAIN bit.
4. [Assessed] Given an unseen capture with several legitimate publishers and a buried abuse, differentiate normal telemetry from a retained-message harvest and from an unauthorized (retained) command injection, justifying each finding by citing the packet type, topic, and RETAIN/return-code fields.
5. [Assessed] Distinguish an authentication failure from an authorization failure in an MQTT intrusion — determining from CONNECT/CONNACK whether the rogue was accepted, and from the PUBLISH/SUBSCRIBE whether the abuse is a missing read or write ACL — and critique a proposed 'reject #-subscriptions' defense by showing how the attacker still succeeds.
6. Given the insecure broker, recommend a layered hardening plan (allow_anonymous false, per-topic read/write ACLs bound to identity, TLS on 8883, retained-message and command-topic controls) and predict how the capture changes after each control.
7. Using Zeek's MQTT analyzer, produce mqtt_connect/publish/subscribe logs and derive detections for anonymous CONNECTs and for '#'/retained-message abuse, naming the log field each alert keys on.

Scope & threat model — read this first

This module teaches you to **read** MQTT and **recognize** its weaknesses on the wire. It does **not** teach you to attack a plant — it deliberately hands you the one thing a real operator-of-harm must earn first: reachability to the broker (or a position on the client↔broker path).

The real ICS/IIoT kill chain (MITRE ATT&CK for ICS): Initial Access → pivot into the OT/IIoT network (Lateral Movement) → recon & topic/asset enumeration (Discovery; Network Sniffing T0842) → **the injection you practice here** (Unauthorized Command Message, T0855) → optional impact/persistence (Manipulation of Control T0831).

The frame-52 command PUBLISH is the **last ~5%**. The enumeration, the L2 adjacency to the broker/client, and the broker access that precede it are the hard 95% — and are out of scope here. See 'What would make this real offense' below.

3. Where this protocol lives — industry & use cases

MQTT rarely appears in isolation — it is the connective tissue of modern telemetry, spanning consumer smart homes up through industrial sensor fleets and cloud IoT platforms. In OT settings it increasingly rides alongside legacy SCADA as plants add sensors and push data to analytics.

Industrial IoT & smart manufacturing. Sensor-to-cloud telemetry for vibration, temperature, energy, and predictive-maintenance data; MQTT (often via Sparkplug B) links edge gateways to historians and dashboards.

Utilities & smart infrastructure. Water/energy telemetry from distributed assets, smart-metering backhaul, and building-management sensor data — exactly the 'plant/tank1/telemetry' pattern in this capture.

Connected products & smart home. Thermostats, locks, trackers, and hubs almost universally speak MQTT to a cloud broker; much public MQTT exposure research comes from this sector.

Connected vehicles, logistics & healthcare. Fleet/asset location, cold-chain monitoring, and device telemetry where MQTT's low overhead suits cellular links.

Typical use cases

- Telemetry ingestion: many sensors PUBLISH readings to per-asset topics; a dashboard SUBSCRIBEs with a wildcard to see them all.
- Command & control: a controller PUBLISHes to a command topic a device is subscribed to (the pattern the attacker abuses in frame 52).
- Presence & health: the Last Will & Testament lets the broker announce a device 'offline' automatically if it drops.
- Store-and-forward: QoS 1/2 and retained messages let intermittently-connected devices exchange data reliably.

4. Protocol anatomy

Every MQTT packet begins with a 2-byte-minimum fixed header: a control packet type in the high nibble of byte 1, flags in the low nibble, and a variable-length 'remaining length' field. Most types then carry a variable header and payload. Wireshark decodes all of this under the 'MQ Telemetry Transport Protocol' tree.

Fixed header

Byte 1 high nibble = packet type (CONNECT=1, CONNACK=2, PUBLISH=3, PUBACK=4, SUBSCRIBE=8, SUBACK=9, PINGREQ=12, PINGRESP=13, DISCONNECT=14). For PUBLISH the low nibble carries DUP, QoS (2 bits), and RETAIN. Then a 1–4 byte Remaining Length.

```
type(4b) | flags(4b) | remaining-length(1–4B)
```

CONNECT variable header & payload

Protocol name 'MQTT', level 4 (=3.1.1), a connect-flags byte (username, password, will retain, will QoS, will flag, clean session), and keep-alive. Payload order: client ID, will topic, will message, username, password.

```
'MQTT' · level · connect-flags · keep-alive · [client-id, will, user, pass]
```

PUBLISH / SUBSCRIBE

PUBLISH carries a topic name, a packet identifier (only if QoS>0), then the payload bytes. SUBSCRIBE carries a packet identifier and one or more (topic filter + requested-QoS) pairs; SUBACK returns a granted-QoS code per filter.

```
topic · [packet-id if QoS>0] · payload
```

Function / packet types you will see

Code	Name	Meaning
1	CONNECT	Client → broker: open a session (carries credentials, will, keep-alive).
2	CONNACK	Broker → client: accept (return code 0) or refuse.
3	PUBLISH	Send a message to a topic (either direction, via the broker).
4	PUBACK	Acknowledge a QoS-1 PUBLISH.
8	SUBSCRIBE	Client → broker: register interest in topic filter(s).
9	SUBACK	Broker → client: granted QoS per requested filter.
12/13	PINGREQ / PINGRESP	Keep-alive heartbeat in both directions.
14	DISCONNECT	Client → broker: clean shutdown of the session.

5. The capture at a glance

A Mosquitto broker (10.10.20.10:1883) serves a small plant. An HMI dashboard (10.10.20.30) subscribes to 'plant/+/telemetry'; a field sensor (10.10.20.7) connects with a Last Will and publishes tank readings, which the broker fans out to the HMI. Then a rogue host (10.10.20.66) connects with no credentials, subscribes to '#' (every topic), receives the leaked telemetry, and publishes a command to 'plant/tank1/command'.

Host / role	Address	Notes
MQTT broker	`10.10.20.10:1883`	Mosquitto broker — the hub every client connects to
HMI / dashboard	`10.10.20.30`	SCADA/HMI subscriber to plant/+/telemetry
Field sensor	`10.10.20.7`	Publishes plant/tank1/telemetry; sets a Last Will & Testament
Rogue host	`10.10.20.66`	Connects anonymously, subscribes to #, injects a command

61 frames · 3 TCP streams · TCP/1883 · subscribe + publish fan-out + anonymous eavesdrop/inject

6. Frame-by-frame walkthrough

Open the matching `.pcap` in Wireshark and follow along; the frame numbers line up exactly.

Frame 1 — 49512 → 1883 [SYN]

`t=0.000s · 10.10.20.30 → 10.10.20.10 · TCP`

The HMI opens a TCP connection to the broker on MQTT's default plaintext port 1883.

Field	Value
Dst port	1883 (MQTT)
Flags	SYN

Why it matters. Port 1883 is unencrypted MQTT. Its secure sibling is 8883 (MQTT over TLS). Seeing 1883 tells you every byte that follows — including passwords — is readable on the wire.

NOTE. Plaintext port. The same conversation on 8883 would be opaque to a sniffer.

Wireshark filter: `tcp.port==1883 && tcp.flags.syn==1`

Frame 4 — Connect Command (hmi-scada-01)

`t=0.002s · 10.10.20.30 → 10.10.20.10 · MQTT`

The HMI's CONNECT. It identifies itself, requests a clean session, sets a 60-second keep-alive, and supplies a username and password — all visible in Wireshark.

Field	Value
Client ID	hmi-scada-01
Clean session	Set
Keep-alive	60 s
Username	hmi_operator
Password	Plant!ntel2024 (cleartext!)

Why it matters. Expand the CONNECT tree in Wireshark and you will read the password in plain ASCII. MQTT credentials are part of the CONNECT payload with no protection of their own.

CRITICAL. Cleartext credential exposure. Anyone on-path harvests hmi_operator / Plant! ntel2024 with no cracking required. TLS (8883) is the fix.

Wireshark filter: `mqtt.msgtype==1`

Frame 6 — Connect Ack (accepted)

`t=0.004s · 10.10.20.10 → 10.10.20.30 · MQTT`

The broker accepts the connection with return code 0 (Connection Accepted). The HMI now has a live session.

Field	Value
Msg type	CONNACK (2)
Return code	0 — Connection Accepted
Session present	0

Why it matters. A return code of 0 means success; codes 1–5 are refusals (bad protocol level, bad credentials, not authorized, etc.). Watching CONNACK codes tells you whether a broker is enforcing anything.

Wireshark filter: `mqtt.msgtype==2`

Frame 8 — Subscribe Request [plant/+/telemetry]

`t=0.006s · 10.10.20.30 → 10.10.20.10 · MQTT`

The HMI subscribes to 'plant/+/telemetry' at QoS 1. The '+' is a single-level wildcard: it matches plant/tank1/telemetry, plant/tank2/telemetry, and so on — but only one level.

Field	Value
Msg type	SUBSCRIBE (8)
Packet id	1
Topic filter	plant/+/telemetry
Requested QoS	1

Why it matters. Wildcards are how one dashboard follows many assets. '+' matches exactly one level; '#' (later) matches everything beneath a point. Scoping subscriptions tightly is both good design and a security control.

Wireshark filter: `mqtt.msgtype==8`

Frame 10 — Subscribe Ack (granted QoS 1)

`t=0.007s · 10.10.20.10 → 10.10.20.30 · MQTT`

The broker grants the subscription at QoS 1 and returns the matching packet identifier.

Field	Value
Msg type	SUBACK (9)
Packet id	1
Granted QoS	1

Why it matters. SUBACK returns a granted-QoS code per requested filter (0/1/2), or 0x80 for failure. A broker that enforced ACLs could refuse here — this one grants everything.

Wireshark filter: `mqtt.msgtype==9`

Frame 15 — Connect Command (field-sensor-07, with Will)

`t=0.361s · 10.10.20.7 → 10.10.20.10 · MQTT`

The field sensor connects. Besides credentials, it registers a Last Will & Testament: if it drops unexpectedly, the broker will publish 'offline' to plant/tank1/status on its behalf.

Field	Value
Client ID	field-sensor-07
Keep-alive	30 s
Username / Password	sensor_svc / s3ns0r-pw (cleartext)
Will topic	plant/tank1/status
Will payload	offline

Why it matters. The Will is MQTT's dead-man's switch — how a fleet knows a device died without polling it. You can read the will topic and payload right in the CONNECT.

CAUTION. Second set of cleartext credentials. Also note: a will payload is attacker-readable intelligence about topic structure.

Wireshark filter: `mqtt.msgtype==1 && mqtt.willtopic`

Frame 19 — Publish [plant/tank1/telemetry] QoS 0

`t=0.364s · 10.10.20.7 → 10.10.20.10 · MQTT`

The sensor publishes a tank reading to plant/tank1/telemetry at QoS 0 (fire-and-forget). The JSON payload — level, temperature, flow — is fully visible.

Field	Value
Msg type	PUBLISH (3)
Topic	plant/tank1/telemetry
QoS	0
Payload	{"level_pct":72.4,"temp_c":21.6,"flow_lpm":11.8}

Why it matters. QoS 0 means at-most-once: no PUBACK, minimal overhead — typical for high-rate telemetry. The payload format is entirely up to the application (here, JSON).

NOTE. Process data in the clear. Over time, captured telemetry reveals plant behavior and setpoints (the reconnaissance value Trend Micro documented at scale).

Wireshark filter: `mqtt.msgtype==3 && mqtt.topic=="plant/tank1/telemetry"`

Frame 21 — Publish → HMI (broker fan-out, QoS 0)

`t=0.366s · 10.10.20.10 → 10.10.20.30 · MQTT`

The broker forwards the sensor's reading to the HMI, which subscribed to a matching filter. The HMI subscribed at QoS 1, but the sensor published at QoS 0 — so the broker delivers at QoS 0, with no packet id and no PUBACK.

Field	Value
Msg type	PUBLISH (3)
Direction	broker → subscriber
QoS	0 (delivered)
Topic	plant/tank1/telemetry

Why it matters. This is the pub/sub magic in two frames: the sensor (frame 19) never addressed the HMI — the broker matched the topic to the subscription and delivered it (frame 21). Delivery QoS is the minimum of the publish QoS and the subscription's max QoS, so a broker may downgrade but never upgrade it.

Wireshark filter: `mqtt.msgtype==3 && ip.dst==10.10.20.30`

Frame 23 — Publish [plant/tank1/telemetry] QoS 1 (id=15)

`t=0.568s` · `10.10.20.7` → `10.10.20.10` · MQTT

The next reading is published at QoS 1 (at-least-once). Because QoS>0 it carries a packet identifier (15); the broker acknowledges it with a PUBACK (frame 25) and — since the HMI also subscribed at QoS 1 — fans it out to the HMI at QoS 1 (frame 27).

Field	Value
Msg type	PUBLISH (3)
Topic	plant/tank1/telemetry
QoS	1
Packet id	15
Payload	{"level_pct":73.1,"temp_c":21.7,"flow_lpm":12.0}

Why it matters. Contrast with frame 19: QoS 1 adds a packet identifier and a guaranteed acknowledgement, at the cost of extra frames. Sensors mix QoS 0 for cheap high-rate telemetry and QoS 1 for readings that must not be lost.

Wireshark filter: `mqtt.msgtype==3 && mqtt.qos==1`

Frame 29 — Publish Ack (id=41)

`t=0.572s` · `10.10.20.30` → `10.10.20.10` · MQTT

The HMI acknowledges the QoS-1 delivery it received from the broker (frame 27) with a PUBACK carrying the same packet identifier, closing the at-least-once loop.

Field	Value
Msg type	PUBACK (4)
Packet id	41

Why it matters. Match packet ids to pair each QoS-1 PUBLISH with its PUBACK. Note the ids differ per hop (15 for sensor→broker, 41 for broker→HMI): each MQTT link manages its own identifiers.

Wireshark filter: `mqtt.msgtype==4`

Frame 31 — Ping Request (keep-alive)

`t=0.874s · 10.10.20.7 → 10.10.20.10 · MQTT`

With no data to send, the sensor sends a PINGREQ to keep its session alive within the 30-second keep-alive window.

Field	Value
Msg type	PINGREQ (12)
Payload	none

Why it matters. PINGREQ/PINGRESP is a 2-byte heartbeat. If the broker hears nothing for 1.5× keep-alive, it considers the client dead and fires its Will. Heartbeats are how brokers detect silent drop-offs.

Wireshark filter: `mqtt.msgtype==12 || mqtt.msgtype==13`

Frame 38 — ⚠️ Connect Command (anonymous) ⚠️ ANOMALY

`t=1.280s · 10.10.20.66 → 10.10.20.10 · MQTT`

A rogue host connects with client ID ‘mqtt-explorer-x’ and NO username or password. Whether this succeeds depends entirely on the broker’s allow_anonymous setting.

Field	Value
Client ID	mqtt-explorer-x
Username	(none)
Password	(none)
Clean session	Set

Why it matters. Compare this CONNECT to frames 4 and 15: the connect flags have no username/password bits set. An anonymous connect is only as safe as the broker’s configuration.

CRITICAL. Anonymous access attempt. Mosquitto 2.0+ defaults to allow_anonymous false, but countless internet-exposed brokers set it true — Avast found ~32,000 brokers with no password at all.

Wireshark filter: `mqtt.msgtype==1 && !mqtt.username`

Frame 40 — ⚠️ Connect Ack (accepted anonymously) ⚠️ ANOMALY

`t=1.281s · 10.10.20.10 → 10.10.20.66 · MQTT`

The broker accepts the anonymous client with return code 0. This broker is misconfigured to allow_anonymous true — the rogue host now has a full session.

Field	Value
Msg type	CONNACK (2)
Return code	0 — Connection Accepted

Why it matters. A return code of 0 to a credential-less CONNECT is the single clearest 'this broker is open' signal you can find in a capture.

CRITICAL. The broker authorized an unauthenticated client. Set allow_anonymous false and require a password_file (or client certificates).

Wireshark filter: `mqtt.msgtype==2 && ip.dst==10.10.20.66`

Frame 42 — ⚠️ Subscribe Request [#] — all topics ⚠️ ANOMALY

`t=1.283s · 10.10.20.66 → 10.10.20.10 · MQTT`

The rogue host subscribes to '#', the multi-level wildcard that matches every topic on the broker. In one request it asks to receive everything the broker relays.

Field	Value
Msg type	SUBSCRIBE (8)
Topic filter	# (multi-level wildcard)
Requested QoS	0

Why it matters. '#' at the root is the classic MQTT eavesdropping move — and a favorite of exposure researchers, because an open broker will happily stream its entire message flow to whoever asks.

CRITICAL. Full-broker eavesdrop. With no topic ACLs, the broker will grant it. This is exactly how public MQTT scans harvest sensitive data (Lundgren, DEF CON 24).

Wireshark filter: `mqtt.msgtype==8 && mqtt.topic=="#"`

Frame 50 — ⚠️ Publish → rogue host (data leaked) ⚠️ ANOMALY

`t=1.439s · 10.10.20.10 → 10.10.20.66 · MQTT`

Because the rogue host subscribed to '#', the broker now forwards the plant's telemetry to it — the attacker passively collects live process data.

Field	Value
Msg type	PUBLISH (3)
Direction	broker → rogue subscriber
Topic	plant/tank1/telemetry
Payload	{“level_pct”:73.6,…}

Why it matters. Trace it: the sensor published (frame 46), and the broker delivered the same message to both the legitimate HMI (frame 48) and the eavesdropper (frame 50). The attacker did nothing but subscribe.

CRITICAL. Confidentiality breach with no exploit — just a subscription. Topic ACLs restricting which clients may read which topics would have blocked this.

Wireshark filter: `mqtt.msgtype==3 && ip.dst==10.10.20.66`

Frame 52 — ⚠️ Publish [plant/tank1/command] — injection ⚠️ ANOMALY

`t=1.641s · 10.10.20.66 → 10.10.20.10 · MQTT`

The rogue host publishes a command to ‘plant/tank1/command’ — {“actuator”:“pump1”,“cmd”:“START”,“valve”:“open”}. Any device subscribed to that command topic would act on it.

Field	Value
Msg type	PUBLISH (3)
Topic	plant/tank1/command
Payload	{“actuator”:“pump1”,“cmd”:“START”,“valve”:“open”}
QoS	0

Why it matters. MQTT has no concept of ‘who is allowed to publish here.’ The broker accepts the write purely because nothing stops it. If an actuator trusts this topic, the attacker just moved physical equipment.

CRITICAL. Unauthorized command injection. Controls: broker ACLs scoping write access per client/topic; authenticate and validate commands at the subscriber; separate telemetry and command brokers/credentials.

Wireshark filter: `mqtt.msgtype==3 && mqtt.topic=="plant/tank1/command"`

Frame 54 — ⚠️ Disconnect ⚠️ ANOMALY

`t=1.642s · 10.10.20.66 → 10.10.20.10 · MQTT`

The rogue host cleanly disconnects, having harvested credentials-free telemetry and injected a command — a full intrusion in a handful of frames.

Field	Value
Msg type	DISCONNECT (14)
Payload	none

Why it matters. A clean DISCONNECT suppresses the client's Will. Reviewing this whole session end-to-end (frames 38–54) is a compact case study in what an open broker allows.

NOTE. The entire rogue session took under half a second and left almost no trace at the application layer — which is why broker-side auth/authorization and network monitoring matter.

Wireshark filter: `mqtt.msgtype==14`

7. Security risks & controls

M1 · Cleartext credentials on port 1883

Severity: CRITICAL, frames 4, 15

- **Risk.** MQTT username/password live in the CONNECT payload with no protection. On plaintext 1883 (frames 4 and 15) anyone on-path reads them directly in Wireshark — here, hmi_operator / Plant!ntel2024 and sensor_svc / s3ns0r-pw.
- **Real-world.** Exposed-broker research (Lucas Lundgren, DEF CON 24, 2016) repeatedly found brokers with weak or sniffable credentials; the protocol offers no credential protection on its own.
- **Technique.** Credential capture / sniffing
- **Control.** Run MQTT over TLS on 8883 so the whole CONNECT — credentials included — is encrypted. Use strong, unique per-client credentials or, better, client certificates. Never expose 1883 beyond a trusted segment.

M2 · No transport encryption — telemetry & topics in the clear

Severity: HIGH, frames 19, 50

- **Risk.** On 1883 every topic name and payload is visible (frames 19, 50). Captured over time, plant telemetry reveals process behavior, setpoints, and topic structure — a reconnaissance goldmine — and messages can be intercepted or tampered with in transit.
- **Real-world.** Trend Micro (2018) observed over 200 million MQTT messages leaking from exposed brokers in a four-month window, including data usable for reconnaissance and lateral movement.
- **Technique.** Interception / passive collection
- **Control.** TLS on 8883 for confidentiality and integrity; segment IoT/OT networks so capture points are limited; avoid putting sensitive data in topic names.

M3 · Anonymous / unauthenticated access

Severity: CRITICAL, frames 38, 40

- **Risk.** If the broker allows anonymous connections, a client with no credentials gets a full session (frames 38–40) and can subscribe and publish like any other.
- **Real-world.** Avast (2018) used Shodan to find roughly 49,000 internet-exposed MQTT brokers, of which about 32,000–33,000 had no password protection at all — smart-home dashboards, locks, and location feeds reachable by anyone.
- **Technique.** Unauthorized access
- **Control.** Set allow_anonymous false and require a password_file or client certificates. Firewall the broker so only known clients can reach it; never expose it to the internet unintentionally.

M4 · No authorization — wildcard eavesdropping

Severity: HIGH, frames 42, 50

- **Risk.** MQTT 3.1.1 defines no access control. Absent broker-side ACLs, any client can subscribe to '#' and receive every message the broker relays (frames 42, 50) — instant, silent confidentiality loss.
- **Real-world.** Subscribing to '#' on open brokers is the standard technique in public MQTT exposure studies; it requires no exploit, only a subscription.
- **Technique.** Eavesdropping / data collection
- **Control.** Enforce per-topic ACLs at the broker (Mosquitto acl_file, or EMQX/HiveMQ equivalents): scope each client to only the topics it needs. Brokers enforce ACLs per message topic, so a client receives nothing outside its grant even if it subscribes to '#'. Combine with authentication (control M3) so ACLs bind to identity — that is what stops the anonymous rogue client, at CONNECT, before any subscription.

M5 · Unauthorized publish / command injection

Severity: CRITICAL, frames 52

- **Risk.** With no write authorization, any connected client can publish to a command topic (frame 52). If an actuator or controller trusts that topic, an attacker can drive physical equipment through the broker.
- **Real-world.** Exposure research has shown open brokers permitting publishes to control/command topics, turning data leaks into potential physical impact.
- **Technique.** Command injection via topic write
- **Control.** Broker ACLs granting write only to authorized publishers per topic; validate and authenticate commands at the subscribing device (don't trust topic membership alone); separate telemetry and command paths (distinct brokers/credentials/segments).

Frame-52 triage — impact is currently notional

As shipped, nothing in the base capture subscribes to `plant/tank1/command` (the HMI takes `plant/+/telemetry`; the sensor's Will is on `.../status`), so the injected command is delivered to no actuator — it proves the broker **accepts** an unauthorized write, not that equipment moved. The Docker lab closes this: a `pump-controller.py` subscriber now acts

on `plant/tank1/command`, so you can watch the injected command actually toggle a simulated pump. Absent that subscriber, treat the impact as **demonstrative**.

What would make this real offense (out of scope in this kit)

A fuller offensive track — deliberately **out of scope** here — would add:

- **Asset/topic enumeration** — reconstruct the topic tree, client-ids, and credentials from a capture or broker probing, instead of being handed the topics.
- **L2 positioning** — ARP-spoof the client↔broker path; this kit assumes that adjacency already exists.
- **TCP-session hijack** — take over a legitimate client's established broker session instead of opening a new anonymous one.
- **Retained-message & $SYS harvest** — pull retained messages and $SYS/# broker metrics for zero-touch recon (exactly the technique in the unseen assessment capture).
- **Sparkplug B command injection** — forge NCMD/DCMD payloads against a Sparkplug edge node — the real IIoT command path, not a bare topic string.
- **Client-id takeover** — reconnect with a live client's id to evict it and assume its session/ACL identity.

8. Hands-on lab

The Docker lab runs a real Mosquitto broker plus paho-mqtt Python publisher/subscriber scripts, so you can reproduce this whole capture live and then harden it. You will start with the insecure default (anonymous, 1883), watch it in Wireshark, then add authentication, ACLs, and TLS and watch the capture change.

Exercise 1. Read a password off the wire

1. Open mqtt_iot_telemetry.pcap in Wireshark.
2. Apply mqtt.msgtype==1 and expand the CONNECT tree on frame 4.

Question. What are the HMI's username and password, and which single control would have prevented you from reading them?

Answer. hmi_operator / Plant!ntel2024, readable in cleartext. Running MQTT over TLS on port 8883 would have encrypted the entire CONNECT, including the credentials.

Exercise 2. Trace one message to two subscribers

1. Filter mqtt.msgtype==3 (all PUBLISH).
2. Follow the third telemetry reading from the sensor through the broker.

Question. Starting at the sensor's publish (frame 46), which frames deliver that same reading, and to whom?

Answer. Frame 46 (sensor→broker) is fanned out to the legitimate HMI in frame 48 and to the rogue eavesdropper in frame 50. One publish, two deliveries — the second is an unauthorized leak.

Exercise 3. Catch the anonymous intruder

1. Filter mqtt.msgtype==1 and compare the connect flags of frames 4, 15, and 38.
2. In the lab container run: zeek -C -r /pcaps/mqtt_iot_telemetry.pcap and open mqtt_connect.log.

Question. Which CONNECT is anonymous, and what does mqtt_connect.log show for it?

Answer. Frame 38 (client mqtt-explorer-x) has no username/password — visible in the CONNECT connect-flags. But do NOT try to catch it by 'empty will fields' in mqtt_connect.log: that log has no username/password column at all, and the legitimate authenticated HMI (hmi-scada-01) also logs empty will fields — so that discriminator both misses and false-positives. Detect anonymous access from the broker's own auth telemetry (Mosquitto's '… as anonymous' / repeated CONNACK rc=5), not Zeek's connect log, which sees only client_id and connect_status, never credentials.

Exercise 4. Harden the broker

1. In the lab, edit mosquitto.conf: set allow_anonymous false, add a password_file, and an acl_file scoping the HMI to read plant/+/telemetry only.
2. Restart the broker and re-run the publisher/subscriber and a '#' subscriber.

Question. After hardening, what happens to the anonymous connect, and how does the capture differ?

Answer. The anonymous CONNECT is refused with CONNACK return code 5 (Not Authorized) — verified in the lab — so the rogue host never connects or subscribes, and the leaked-telemetry frames disappear. The ACL is defense-in-depth: Mosquitto enforces it per message topic (a client only receives topics its rule grants) rather than by failing the '#' SUBACK, so scope credentials tightly instead of relying on rejecting the wildcard itself. Adding TLS on 8883 further hides credentials and payloads entirely.

9. O*NET personas & career pathways

This module is framed around real O*NET occupational personas — the subject-matter voices that shaped it and the people whose work it maps to (see the split below: skills you practice vs. who this protects).

15-1212.00 — Information Security Analysts *(SME voice (Bright Outlook))*

> "I pull the pcap, read the cleartext creds in frame 4, and write the alert for anonymous CONNECTs."

O*NET tasks: 'Encrypt data transmissions … to keep out tainted digital transfers' and 'Monitor use of data files and regulate access.' Tool list names Wireshark and IDS/IPS — the analyst who triages this broker.

15-1299.05 — Information Security Engineers *(Builds the defenses (Bright Outlook))*

> "I stand up the hardened broker: TLS on 8883, password_file, and per-topic ACLs."

O*NET tasks: ‘Develop or install software, such as firewalls and data encryption programs’ and ‘Identify security system weaknesses, using penetration tests.’ The engineer who implements M1–M5’s controls.

15-1241.00 — Computer Network Architects *(Segmentation & packet analysis (Bright Outlook))*

> “I decide where the broker lives and who can reach 1883/8883 — and I read the packets to prove it.”

The only occupation of this set whose O*NET tool list names ‘Packet analysis software’ as a discrete entry; owns the segmentation that keeps a broker off the open internet.

17-2071.00 — Electrical Engineers *(IIoT system design (Bright Outlook))*

> “As I add sensors to the plant, I choose whether telemetry and commands share a broker — a security decision.”

O*NET tools include SCADA/PLC/HMI software; the persona integrating MQTT sensors alongside legacy control and deciding the topic/command architecture.

15-1299.04 — Penetration Testers *(Adversary emulation (Bright Outlook))*

> “I demonstrate the frame 38→52 intrusion in a lab so the org funds ACLs and TLS.”

O*NET tasks: ‘Develop security penetration testing processes (wireless, data networks, telecommunications).’

25-9031.00 — Instructional Coordinators *(Curriculum author)*

> “I built this module from a verified capture, mapping each objective to a real occupation.”

O*NET tasks: ‘Interview subject-matter experts … to develop instructional content’ and keep training ‘technologically current.’

What you practice → who does this work

These occupations do packet and detection analysis (or implement the controls) as their actual job — the skills this kit rehearses.

Skill you practice in this kit	O*NET	The real work it maps to
Identify MQTT control packets, extract CONNECT details (including reading cleartext credentials), and trace pub/sub fan-out — distinguishing a live PUBLISH from a delivered retained message by the RETAIN flag	15-1212.00	Information Security Analysts — monitor use of data files, regulate access, and analyze traffic to detect intrusions

Skill you practice in this kit	O*NET	The real work it maps to
Decide where the broker lives and who may reach 1883/8883, and read the packets to prove the segmentation holds	15-1241.00	Computer Network Architects — design data-communication networks ('packet analysis software' is a named O*NET tool)
Stand up the hardened broker: allow_anonymous false, password_file, per-topic read/write ACLs, TLS on 8883, and retained-message/command-topic controls	15-1299.05	Information Security Engineers — develop or install firewalls and data-encryption software; identify weaknesses using penetration tests
Safely emulate the anonymous harvest-and-inject intrusion (including the unseen retained-message abuse) to justify the controls	15-1299.04	Penetration Testers — develop security penetration-testing processes across data networks

Context: who this protects

These roles operate, maintain, design, or authored the systems under analysis — not skills the learner performs here.

Occupation	O*NET	Why they are in the room
Electrical Engineers	17-2071.00	Decide whether telemetry and commands share a broker — a design/procurement security decision — and must require the controls the analyst specifies.
Instructional Coordinators	25-9031.00	Authored this module from the verified capture — the author's own occupation, not a learner skill.
Plant-floor operators / HMI users	(no single O*NET code)	The people whose process view and equipment the analysis protects; the retained command injection targets their actuators.

10. References & sources

- OASIS — MQTT Version 3.1.1 Standard
- Eclipse Mosquitto — broker & mosquitto.conf (allow_anonymous, acl_file, TLS)

- Zeek — built-in MQTT analyzer & logs
- Lucas Lundgren & Neal Hindocha — DEF CON 24: ‘Light-Weight Protocol! Serious Equipment! Critical Implications!’
- Avast (M. Hron, 2018) — exposed MQTT in smart homes
- Trend Micro (2018) — ‘The Fragility of Industrial IoT’s Data Backbone’
- NIST SP 800-82 Rev. 3 — Guide to Operational Technology (OT) Security
- OWASP Internet of Things Project
- O*NET — Information Security Analysts 15-1212.00
- O*NET — Information Security Engineers 15-1299.05